LOGO AI Task and Prompt Pack (copy-paste version)

Paste Block A at the start of every task chat, then PROJECT_STATE.md, then the task PROMPT plus the items under 'You provide'.

Block A

```
PROJECT: B.Tech major project "LOGO: video text spotting" = re-implementation of
PIPELINE (each stage reads the previous stage's files from disk): frames -> det -
ENV: Python 3.11, PyTorch 2.1.2+cu121, NumPy 1.26.4, one RTX 3050 6GB laptop GPU
RULES:
1) Follow the CONTRACT below exactly. Never rename files, JSON keys, config keys
2) Write complete runnable code: no stubs, no TODO placeholders, no invented thir
3) Every new module gets a pytest in tests/ that runs on CPU with synthetic data
4) Seeds come from the config. Every run writes runs/<exp_id>/{config.yaml,log.tx
5) Never use the test split unless the script is called with --final.
6) Keep answers short: no tutorials, no restating my prompt.
RESPONSE FORMAT: (1) files created/changed; (2) full code for each new file, or a
CONTRACT
Paths: data/raw/<ds>/ ; data/frames/<ds>/<video>/%06d.jpg (frame ids 1-based) ; d
GT json: {"video","dataset","width","height","num_frames","frames":{"1":[{"track_
det json: {"video","dataset","frames":{"1":[{"quad":[8 floats],"score":float}]}}
lsc json: det objects plus {"text":str,"text_conf":float,"p_eos":float,"lang_hard
fused json: lsc objects plus {"score_fused":float}.
tracks: <video>.jsonl, one line per box: {"frame":int,"track_id":int,"quad":[8],"
metrics.json: {"det":{"precision","recall","f"}, "track":{"MOTA","MOTP","IDF1","M
Package: src/logo/{utils,data,detector,lsc,glyph,vpmm,tracking,scoring,voting,eva
Functions: logo.utils.io: load_json(p), save_json(o,p), load_yaml(p), set_seed(s)
logo.data.schema: load_gt(path)->dict ; list_videos(ds,split)->list[str]
logo.eval.det_eval.evaluate_detection(gt_dir,pred_dir,videos,iou=0.5)->dict
logo.eval.mot_eval.evaluate_mot(gt_dir,track_dir,videos,spotting,iou=0.5)->dict
logo.lsc.roi_rotate.crop_quad(img,quad,out_hw=(32,128))->np.ndarray uint8 HxWx3
logo.scoring.fusion.fuse(det,lang,mode,w=0.5)->float   (mode "hard" or "soft")
logo.tracking.simple_tracker.run_tracker(fused,cfg)->list[dict]
logo.vpmm.module.VPMM(c,d): forward(quads[N,4,2], feats[N,c,d], img_wh[N,2]) -> [
logo.voting.track_vote.vote_tracks(tracks,cfg)->list[dict]
Config keys (yaml): exp_id, dataset, split, seed, detector{}, lsc{ckpt,det_thr:0.
```

PROJECT_STATE.md template

```
# PROJECT_STATE.md  (keep under 40 lines)
## Done (task ids + one line each)
P01 repo skeleton, io utils
## Decisions / deviations from the paper
- (example) detector: Plan A PP-YOLOE-R small, input 1024x576
## Interfaces added beyond the CONTRACT (name, signature, file)
-
## Open problems
-
## Next task
P02
```

Recovery prompts

Fix an error (cheapest)

```
TASK: fix this error only. Do not refactor. Rules and CONTRACT from Block A still
FILE (only the relevant function):
<paste function>
ERROR (last 30 lines):
<paste trace>
Give a minimal unified diff and the command to re-run.
```

Tests pass but numbers look wrong

```
TASK: diagnose, do not rewrite. Expected: <what you expected>. Got: <metrics>. Gi
```

AI broke the CONTRACT

```
You changed <name> which Block A forbids. Revert to the CONTRACT signature/keys a
```

Resume in a fresh chat

```
<paste Block A> <paste PROJECT_STATE.md> TASK: continue with <task id>. Before co
```

Tasks

P01 - Repo skeleton and shared utilities

- Plan card: T1.1 | Env: logo-torch | Depends on: none
- You provide: Nothing (Block A only).

```
TASK P01. Create the repository skeleton exactly as in the CONTRACT:
- src layout with every package under src/logo (utils, data, detector, lsc, glyph
- .gitignore (data/, runs/, *.ckpt, *.pth, *.pdparams, __pycache__, .venv, LaTeX
- Implement logo.utils.io with the five functions (logger writes to console and t
- tests/test_io.py covering all five functions.
Do not add any other modules.
```

You run:
uv pip install -e .
pytest -q

Accept when: pytest passes; python -c "import logo" works; repo pushed to GitHub.

P02 - Environment check and setup notes

- Plan card: T1.2 | Env: logo-torch (+ logo-paddle notes) | Depends on: P01
- You provide: Your OS (Windows or WSL2), NVIDIA driver version (nvidia-smi), and the output of your current torch check.

```
TASK P02. (a) Write tools/check_env.py: print Python, torch, CUDA availability, G
(b) Write docs/env_setup.md using uv throughout (uv venv --python <version>, uv p
(c) Add a 'Plan B' section: a PyTorch rotated detector (Ultralytics YOLO OBB) ins
My OS/driver: <paste>
```

You run:
python tools/check_env.py
then follow docs/env_setup.md for logo-paddle (time-box 1 day)

Accept when: check_env passes in logo-torch AND (paddle run_check passes OR Plan B recorded in docs/decisions.md).

P03 - Dataset inventory

- Plan card: T1.3 | Env: logo-torch | Depends on: P01
- You provide: Output of a directory listing (2 levels) of data/raw/icdar2013v, icdar2015v and dstext (Windows: tree /F, depth limited, or ls -R | head -60).

```
TASK P03. Write tools/inventory.py --root data/raw. For each dataset folder (icda
Directory listings: <paste>
Expected counts (from papers): ICDAR2013 Video 28 videos; ICDAR2015 Video 49 vide
```

You run:
python tools/inventory.py --root data/raw

Accept when: counts match expectations or every difference is explained in docs/data_report.md.

P04 - Unified annotation schema and parsers

- Plan card: T1.4 | Env: logo-torch | Depends on: P03
- You provide: First 40 lines of ONE annotation file per dataset (three samples). Without them the AI will guess the format; do not skip.

```
TASK P04. Write logo.data.parsers (one parser per dataset: parse_icdar2013v, pars
Read the annotation samples below and write parsers for exactly those formats. If
Tests: use tiny synthetic annotation snippets written to a temp dir, assert quad
Samples: <paste three samples>
```

You run:
python tools/convert_annotations.py --root data/raw --out data/processed
pytest -q tests/test_parsers.py

Accept when: all three datasets convert with no error; summary printed and saved to docs/data_report.md. Also confirm the DONT-CARE rule in each dataset's evaluation readme and record it.

P05 - Frames, GT visualizer, statistics, splits

- Plan card: T1.5 + T1.6 | Env: logo-torch | Depends on: P04
- You provide: Nothing, but have a few videos ready.

```
TASK P05. Write: (1) tools/extract_frames.py (OpenCV, JPEG quality 90, names %06d
```

You run:
python tools/extract_frames.py --ds icdar2015v
python tools/viz_gt.py --ds icdar2015v --video <id> --out runs/viz.mp4
python tools/dataset_stats.py
python tools/make_splits.py

Accept when: frame count equals annotated range for 3 videos per dataset; you watched the overlay videos and boxes line up; splits committed (milestone M2).

P06 - Evaluation harness (detection, tracking, spotting)

- Plan card: T2.1 | Env: logo-torch | Depends on: P04
- You provide: Nothing (py-motmetrics is installed). This is the most important module: review it carefully.

```
TASK P06. Implement logo.eval.det_eval and logo.eval.mot_eval plus tools/evaluate
Details: polygon IoU with shapely (cache polygons; prefilter by axis-aligned box
det mode: precision, recall, f (percent) from greedy one-to-one matching by IoU,
track mode: py-motmetrics accumulator with distance 1-IoU, pairs with IoU below t
spot mode: same, but a pair is only allowed when normalized texts are equal (lowe
Also write per-video counts into metrics.json["per_video"] exactly as in the CONT
tests/test_eval.py (synthetic): GT used as prediction gives 100 on every metric;
```

You run:
pytest -q tests/test_eval.py
python tools/evaluate.py --gt_dir data/processed/icdar2015v --pred_dir <gt-as-pred dir> --videos splits/icdar2015v_val.txt --mode spot --out /tmp/m.json

Accept when: all five sanity tests pass; running on GT-as-prediction for one real video returns 100. If the official challenge scripts are available, compare on one video and note differences in docs/decisions.md.

P07A - Detector inference to det.json (Plan A: PP-YOLOE-R, Paddle)

- Plan card: T2.2 | Env: logo-paddle | Depends on: P02, P04
- You provide: Paste PaddleDetection tools/infer.py (or deploy/python/infer.py) and the rotate config you plan to use (configs/rotate/ppyoloe_r/...). The AI must use the API in those files.

```
TASK P07A. Write src/logo/detector/infer_paddle.py: load a PP-YOLOE-R (small) che
Pasted files: <infer.py> <config>
```

You run:
python -m logo.detector.infer_paddle --config configs/E2-det.yaml --split val --max_videos 1

Accept when: det.json written for one video; a drawn overlay (use tools/viz_det.py if you ask for it) shows boxes on text; timing.json present.

P07B - Detector inference to det.json (Plan B: YOLO OBB, PyTorch)

- Plan card: T2.2 (Plan B only) | Env: logo-torch | Depends on: P02, P04
- You provide: Use this ONLY if P02 decided Plan B. Say which Ultralytics version you

installed (pip show ultralytics).

```
TASK P07B (Plan B). Write src/logo/detector/infer_obb.py using the Ultralytics or
```

You run:
python -m logo.detector.prepare_obb_data --config configs/E2-det.yaml
python -m logo.detector.infer_obb --config configs/E2-det.yaml --split val --max_videos 1

Accept when: same as P07A; deviation recorded in docs/decisions.md.

P08 - Detector training data, fine-tune and export (Plan A)

- Plan card: T2.3 to T2.5 | Env: logo-paddle (training may run on Kaggle/Colab or a lab GPU) | Depends on: P07A
- You provide: The training config you will modify and the dataset-format section of the PaddleDetection rotate docs (paste the relevant lines).

```
TASK P08. (1) Write tools/gt_to_dota.py converting data/processed GT (non-dont_ca
Pasted: <train config> <dataset docs lines>
```

You run:
bash scripts/train_detector.sh --smoke
bash scripts/train_detector.sh
python tools/export_all_dets.py --exp E2-det-v1
python tools/evaluate.py --mode det ... (validation F)

Accept when: smoke run finishes without OOM; validation detection F recorded; det.json exists for every video (milestone M3).

P09 - RoI-Rotate crop utility

- Plan card: T3.1 | Env: logo-torch | Depends on: P04
- You provide: Nothing.

```
TASK P09. Implement logo.lsc.roi_rotate.crop_quad(img, quad, out_hw=(32,128)) and
```

You run:
pytest -q tests/test_roi.py
python tools/crop_grid.py --ds icdar2015v --video <id> --out runs/crops.png

Accept when: tests pass; the crop grid shows readable, upright text for oriented words.

P10 - ABINet wrapper and baseline recognition accuracy

- Plan card: T3.2 | Env: logo-torch | Depends on: P09
- You provide: Clone the official PyTorch ABINet into third_party/ABINet and download its pretrained weights. Paste demo.py, the model config yaml and the CharsetMapper class (from utils.py).

```
TASK P10. Write logo.lsc.abinet_wrapper.ABINetRecognizer with: __init__(ckpt, dev
Pasted: <demo.py> <config> <CharsetMapper>
```

You run:
python tools/eval_recognition.py --ds icdar2015v --split val --ckpt <abinet pretrained>

Accept when: word accuracy number recorded in docs/baseline_report.md; batch inference runs inside 6 GB; c and d recorded in PROJECT_STATE.md.

P11 - LSC dataset builder (paper Eq. 1 to 3)

- Plan card: T3.3 | Env: logo-torch | Depends on: P08 (or P07B), P09
- You provide: Nothing.

```
TASK P11. Write tools/build_lsc_dataset.py. Positives: crops (crop_quad) of GT in
```

You run:
python tools/build_lsc_dataset.py --exp E2-det-v1 --split train
python tools/build_lsc_dataset.py --exp E2-det-v1 --split val

Accept when: stats.json saved; you opened 50 random negatives and most are background or text-like, and no negative lies on obvious text.

P12 - Fine-tune the LSC

- Plan card: T3.4 | Env: logo-torch | Depends on: P10, P11
- You provide: Paste the ABINet repo's training entry (main.py / Learner, loss construction) and the fine-tuning yaml.

```
TASK P12. Write src/logo/lsc/train_lsc.py (and configs/E3-lsc.yaml) to fine-tune
Pasted: <training entry> <loss code> <yaml>
```

You run:
python -m logo.lsc.train_lsc --config configs/E3-lsc.yaml --smoke
python -m logo.lsc.train_lsc --config configs/E3-lsc.yaml

Accept when: validation text/background accuracy and word accuracy recorded in runs/E3-lsc-v1; best checkpoint saved.

P13 - LSC inference, hard fusion, detection check

- Plan card: T3.5 | Env: logo-torch | Depends on: P12
- You provide: Nothing.

```
TASK P13. (1) logo.lsc.run_lsc: for each video, read det.json, keep boxes above l
```

You run:
python -m logo.lsc.run_lsc --config configs/E3-lsc.yaml
python -m logo.scoring.run_fuse --config configs/E3-lsc.yaml
python tools/evaluate.py --mode det ...

Accept when: precision rises after re-scoring (paper trend); rescore panel PNG saved; tag v0.3-detector.

P14 - Glyph pseudo-labels (K-means)

- Plan card: T4.1 | Env: logo-torch | Depends on: P11
- You provide: Nothing.

```
TASK P14. Write logo.glyph.pseudo_labels.kmeans_mask(crop_rgb) -> float mask [H,W
```

You run:
python tools/make_glyph_masks.py --split train
python tools/mask_grid.py --split train --out runs/masks.png

Accept when: masks exist for all positive training crops; the grid shows mostly plausible masks, failures noted.

P15 - Glyph branch with MSE loss (time-boxed)

- Plan card: T4.2 | Env: logo-torch | Depends on: P12, P14
- You provide: Paste the ABINet vision backbone (ResNet) class so feature taps can be added, and the shapes of its three feature levels.

```
TASK P15. Add logo.glyph.branch.GlyphHead per paper Eq. 5: inputs backbone featur
```

You run:
python -m logo.lsc.train_lsc --config configs/E4-lsc-glyph.yaml --smoke

python tools/compare_glyph.py --base E3-lsc-v1 --glyph E4-lsc-glyph-v1

Accept when: comparison table saved; keep the better checkpoint as lsc.ckpt for later tasks and record the decision.

P16 - Baseline tracker (ByteTrack-style)

- Plan card: T4.3 | Env: logo-torch | Depends on: P13
- You provide: Nothing.

```
TASK P16. Implement logo.tracking.simple_tracker.run_tracker(fused, cfg) and a CL
```

You run:
pytest -q tests/test_tracker.py
python -m logo.tracking.run_track --config configs/E4-base.yaml

Accept when: tracks.jsonl for all val videos; one overlay clip with ids watched (tools/viz_tracks.py if you ask for it).

P17 - Pipeline runner and first ablation rows

- Plan card: T4.4 | Env: logo-torch | Depends on: P13, P16, P06
- You provide: Nothing.

```
TASK P17. Write scripts/run_pipeline.py --config <yaml> --stage det,lsc,fuse,trac
```

You run:
python scripts/run_pipeline.py --config configs/E4-base.yaml --stage lsc,fuse,track,eval
python scripts/run_pipeline.py --config configs/E4-lsc.yaml --stage lsc,fuse,track,eval
python tools/compare_runs.py E4-base E4-lsc

Accept when: one command regenerates both rows; Baseline+LSC beats Baseline on ICDAR2015 val (paper trend), or you record why not.

P18 - VPMM module

- Plan card: T4.5 | Env: logo-torch | Depends on: P01
- You provide: The c and d values from P10.

```
TASK P18. Implement logo.vpmm.module.VPMM(c, d) per paper Eq. 7 and 8, plus logo.
```

You run:

pytest -q tests/test_vpmm.py

Accept when: tests pass; module runs on a batch of cached LSC features from P13 without error.

P19 - GoMatching LST-Matcher study and adapter plan

- Plan card: T5.1 | Env: logo-torch | Depends on: P18
- You provide: Clone GoMatching into third_party/GoMatching. Paste the LST-Matcher module file(s), its config, and the loss/training step code (search the repo for 'LST', 'matcher', 'ST-Matcher', 'LT-Matcher').

```
TASK P19 (analysis, little code). Read the pasted GoMatching files and answer: (1
Pasted: <matcher files> <config> <loss/training code>
```

You run:
pytest -q tests/test_lst_adapter.py (synthetic tensors, CPU)

Accept when: adapter plan committed to docs/decisions.md; adapter forward/backward runs on random tensors on CPU.

P20 - Feature cache for LST-Matcher training

- Plan card: T5.2 | Env: logo-torch | Depends on: P13, P18, P19
- You provide: Nothing.

```
TASK P20. Write tools/build_lst_cache.py: for each training video, read fused jso
```

You run:
python tools/build_lst_cache.py --exp E3-lsc-v1 --split train
pytest -q tests/test_cache.py

Accept when: cache exists for all train videos; one batch from the loader prints expected shapes; disk size noted.

P21 - Train the LST-Matcher

- Plan card: T5.3 | Env: logo-torch (GPU) | Depends on: P20
- You provide: Paste the optimizer/scheduler part of GoMatching's training script for reference, and your VRAM use from a smoke run if it fails.

```
TASK P21. Write src/logo/tracking/train_lst.py and configs/E5-lst.yaml: train LST
```

You run:
python -m logo.tracking.train_lst --config configs/E5-lst.yaml --smoke
python -m logo.tracking.train_lst --config configs/E5-lst.yaml

Accept when: validation IDF1 beats the P16 baseline. Cut-line: if not working by the sprint-5 deadline use fallback P21F.

P21F - Fallback tracker: ByteTrack-style + VPMM embedding

- Plan card: T5.3 fallback | Env: logo-torch | Depends on: P18, P16
- You provide: Use only if P21 is blocked.

```
TASK P21F (fallback). Extend simple_tracker with tracker.type 'embed': train a sm
```

You run:
python -m logo.tracking.train_embed --config configs/E5-embed.yaml
python scripts/run_pipeline.py --config configs/E5-embed.yaml --stage track,eval

Accept when: IDF1 improves over the P16 tracker or the result is documented; story stays valid because the paper's own ablation uses ByteTrack.

P22 - LST inference, thresholds, trajectory filtering

- Plan card: T5.4 | Env: logo-torch | Depends on: P21 (or P21F)
- You provide: Nothing.

```
TASK P22. Add tracker.type 'lst' to logo.tracking.run_track (uses LSTAdapter.asso
```

You run:
python scripts/run_pipeline.py --config configs/E5-lst.yaml --stage track,eval
python tools/sweep_threshold.py --config configs/E5-lst.yaml --split val

Accept when: tracking and spotting metrics plus the sweep curve saved.

P23 - Baseline freeze and efficiency profile

- Plan card: T5.5 | Env: logo-torch | Depends on: P22
- You provide: Nothing.

```
TASK P23. Write tools/freeze_baseline.py: (1) run the four ablation configs (Base
```

You run:

python tools/freeze_baseline.py --split val
git tag v1.0-base

Accept when: MVP GATE: baseline report exists, reproducible within +-0.3. Reproduction success = same ordering of component gains as the paper, or a documented reason.

P24 - Error analysis on the frozen baseline

- Plan card: T6.1 | Env: logo-torch | Depends on: P23
- You provide: Nothing.

```
TASK P24. Write tools/error_analysis.py using per-frame matching at IoU 0.5 again
```

You run:
python tools/error_analysis.py --exp E5-lst-v1 --split val

Accept when: taxonomy table and gallery committed; top-2 error types named and used to justify P25 and P26.

P25 - C2: soft confidence-aware language re-scoring

- Plan card: T6.2 | Env: logo-torch | Depends on: P24
- You provide: Nothing.

```
TASK P25. Implement mode 'soft' in logo.scoring.fusion.fuse: lang_soft = 1 - p_eo
```

You run:
pytest -q tests/test_fusion.py
python tools/sweep_fusion.py --config configs/E5-lst.yaml --split val

Accept when: CSV + plots saved; you can state whether soft improves a metric or flattens the MOTA/IDF1 trade-off. Fix w and thr on val BEFORE any final run.

P26 - C3: track-level text voting and filtering

- Plan card: T6.3 | Env: logo-torch | Depends on: P25
- You provide: Nothing.

```
TASK P26. Implement logo.voting.track_vote.vote_tracks(tracks, cfg) (tracks = lis
```

You run:
pytest -q tests/test_voting.py

python tools/sweep_voting.py --config configs/E6-vote.yaml --split val

Accept when: spotting MOTA/IDF1 and number of wrong-transcription boxes reported with vs without voting.

P27 - Ablation runner and bootstrap confidence intervals

- Plan card: T6.4 + T6.5 | Env: logo-torch | Depends on: P26
- You provide: Nothing.

```
TASK P27. (1) tools/run_ablation.py reads configs/ablation.yaml (list of named co
```

You run:
python tools/run_ablation.py --config configs/ablation.yaml --split val
python tools/bootstrap_ci.py --a E5-lst-v1 --b E6-all-v1

Accept when: complete ablation table; CI table saved; the report says 'improves' only where the interval excludes zero.

P28 - Final runs and efficiency table

- Plan card: T7.1 + T7.2 | Env: logo-torch (+ GPU for DSText if available) | Depends on: P27
- You provide: Your frozen best config names and chosen w, thr, tau, min_len.

```
TASK P28. Write scripts/final_eval.py --config <best> --final: runs the full pipe
```

You run:
python scripts/final_eval.py --config configs/FINAL.yaml --final

Accept when: FINAL metrics for each dataset; any skipped dataset listed as a limitation.

P29 - Tables and figures generated by script

- Plan card: T7.3 + T7.5 | Env: logo-torch | Depends on: P28
- You provide: Nothing.

```
TASK P29. Write tools/make_tables.py reading runs/*/metrics.json and writing pape
```

You run:
python tools/make_tables.py
python tools/make_figures.py

Accept when: all tables/figures regenerate with one command and open correctly.

P30 - Demo

- Plan card: T7.4 | Env: logo-torch (+ paddle for Plan A detector) | Depends on: P28
- You provide: Nothing.

```
TASK P30. Write scripts/demo.py --video in.mp4 --out out.mp4 [--config FINAL.yaml
```

You run:
python scripts/demo.py --video samples/clip.mp4 --out runs/demo.mp4

Accept when: runs end to end on a video the system has never seen; three backup output clips saved.

P31 - README and reproducibility guide

- Plan card: T8.3 | Env: any | Depends on: P29
- You provide: Your final results table (paste) and checkpoint links.

```
TASK P31. Write README.md: one-paragraph overview, pipeline figure placeholder pa
Results + links: <paste>
```

You run:
a teammate follows the README from a clean clone

Accept when: teammate reproduces one table from scratch following only the README.

P32 - IEEE paper skeleton and first draft

- Plan card: T8.4 | Env: Overleaf / any | Depends on: P29
- You provide: Paste docs/baseline_report.md, the generated tables (.md versions) and your bullet list of contributions.

```
TASK P32. Create paper/main.tex using IEEEtran (conference mode) with sections: A
Pasted: <baseline_report> <tables> <contributions>
```

You run:
compile in Overleaf; then you verify every claim and number

Accept when: draft compiles; no number without a source file; every reference checked against the real paper by you.

P33 - Project report and final presentation drafts

- Plan card: T8.1 + T8.2 | Env: any | Depends on: P29, P32
- You provide: Paste docs/baseline_report.md, docs/decisions.md and the table .md files; your college report template if any.

```
TASK P33. Produce (1) a project report outline with full draft text for: Introduc
Pasted: <files> <college template>
```

You run:
review with your guide

Accept when: guide review done; every number traces to a run.